

PROJECT REPORT – MODERN APPLICATION DEVELOPMENT I

AUTHOR:

NAME: NIDA FARHEEN C M

ROLL NO: 24F1001163

EMAIL: 24F1001163@DS.STUDY.IITM.AC.IN

Hey. I'm a third year undergraduate student studying Computer Science at College of Engineering, Trivandrum. I'm also pursuing data science(distance) at IIT Madras. I enjoy web development and I am actively learning how to build applications using modern frontend tools.

DESCRIPTION:

The project I did was about building a role-based parking lot management web application where Admins can manage parking spaces and Users can book slots according to their convenience. The app allows real-time viewing and reservation of parking spots.

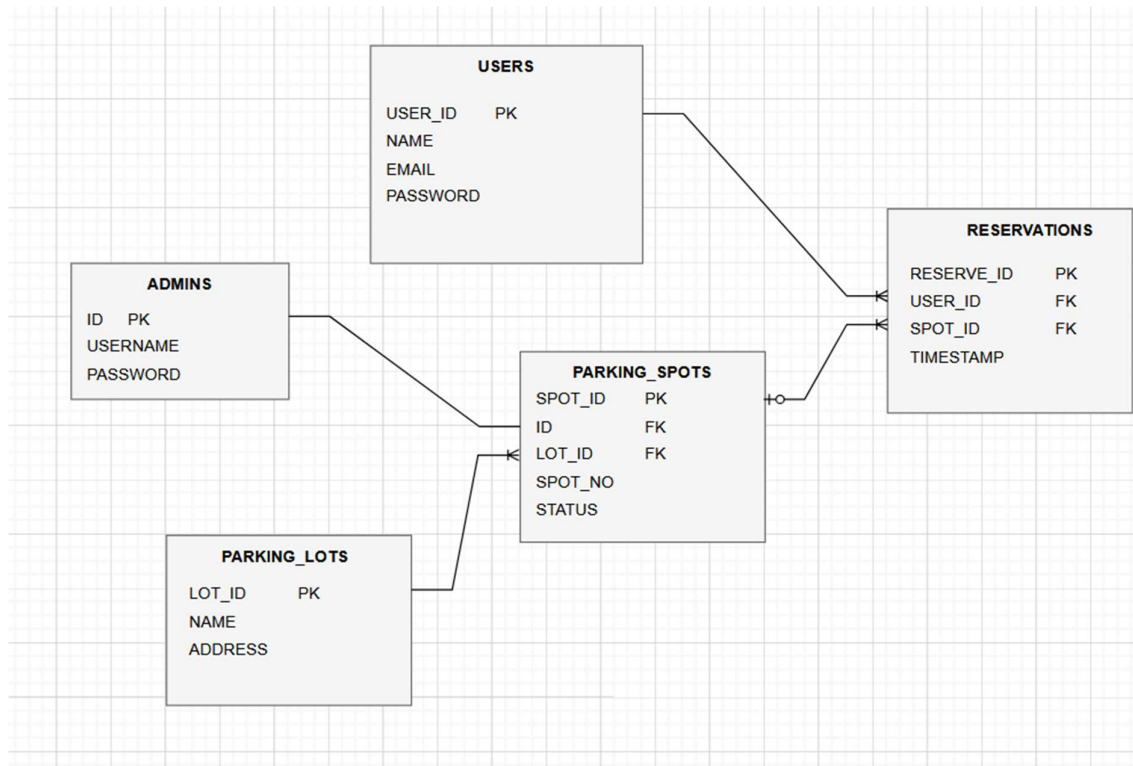
AI/LLM Usage: ~30% – Used ChatGPT for understanding Flask, generating code structure etc.

How I approached the problem: I approached the problem by designing a schema to represent the relationships between admin, users, parking lots, spots, and reservations. Then I built the (Flask) app structure, created routes for user interactions, and used Jinja2 templates to dynamically render pages. I added session based login, form validation, and logic to prevent double booking along with a logout option.

TECHNOLOGIES USED:

- Flask: Core web framework
- Jinja2: For rendering dynamic HTML pages
- SQLite: Lightweight database
- HTML/CSS/Bootstrap: For UI and responsiveness
- Flask Extensions

DB SCHEMA DESIGN:



API DESIGN:

APIs were created for:

- Getting list of parking lots and spots
- Reserving a parking spot
- Admin operations like adding lots/spots

APIs were implemented as Flask routes using `@app.route` with GET and POST methods. Data exchanged via form submissions or query parameters. The API structure is defined in a separate `api.yaml` file for OpenAPI compliance.

API RESOURCE ENDPOINT:

ROUTE DESCRIPTION	END POINT	METHOD
/	Home page	GET
/admin	Admin dashboard	GET
/admin/add_lot	Add new lot	POST
/lots	To display parking lots	GET
/reserve/<spot_id>	Reserve a parking spot	POST
/admin/add_spot	Add new parking spot	POST
/admin/reservations	Display all reservations	GET

ARCHITECTURE AND FEATURES:

App is structured in a modular way:

- app.py contains all routes and logic
- Templates: Located in the templates/ folder, written using Jinja2
- Static Files: CSS and Bootstrap files stored in static/
- Database: Programmatically initialized on first run using SQLite

Features Implemented:

- Admin login and dashboard
- Admin can add parking lots and spots
- Admin can delete a parking lot only if all spots are empty.
- Admin can view available/occupied spots with charts showing parking summary.
- Spot availability updated in real-time
- Users can view available spots and reserve them
- Responsive and interactive
- Basic session-based login and access control

Additional Features:

Error messages and edge-case handling are done along with the automatic database creation. Codebase is organized with clear logic separation.

VIDEO LINK:

<https://drive.google.com/file/d/1KTJlxzMQvuGzCDx6UnC4AZCzqd6CrqUl/view?usp=sharing>